클러스터를 추가합니다

Astra Control Service

NetApp
March 27, 2024

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-service/aws/get-started/install-astra-connector.html on March 27, 2024. Always check docs.netapp.com for the latest.

목차

Astra Control Service에 클러스터를 추가합니다 1
프라이빗 클러스터용 Astra Connector를 설치합니다 1
공급자 관리 클러스터를 추가합니다 4

Astra Control Service에 클러스터를 추가합니다

환경을 설정한 후에는 Kubernetes 클러스터를 생성하고 Astra Control Service에 추가할 준비가 된 것입니다. 이를 통해 Astra Control Service를 사용하여 클러스터의 애플리케이션을 보호할 수 있습니다.

Astra Control Service에 추가해야 하는 클러스터의 유형에 따라 클러스터를 추가하려면 다른 단계를 사용해야 합니다.

- "공용 공급자 관리 클러스터를 Astra Control Service에 추가합니다": 다음 단계를 수행하여 공용 IP 주소가 있고 클라우드 공급자가 관리하는 클러스터를 추가할 수 있습니다. 클라우드 공급자의 서비스 주 계정, 서비스 계정 또는 사용자 계정이 필요합니다.
- "Astra Control Service에 프라이빗 공급자 관리 클러스터를 추가합니다": 다음 단계를 수행하여 프라이빗 IP 주소가 있고 클라우드 공급자가 관리하는 클러스터를 추가할 수 있습니다. 클라우드 공급자의 서비스 주 계정, 서비스 계정 또는 사용자 계정이 필요합니다.
- "공용 자가 관리 클러스터를 Astra Control Service에 추가합니다": 다음 단계를 사용하여 공용 IP 주소가 있고 조직에서 관리하는 클러스터를 추가할 수 있습니다. 추가하려는 클러스터에 대해 kubecononfig 파일을 생성해야 합니다.
- "Astra Control Service에 프라이빗 자체 관리 클러스터를 추가합니다": 다음 단계를 사용하여 개인 IP 주소가 있고 조직에서 관리하는 클러스터를 추가할 수 있습니다. 추가하려는 클러스터에 대해 kubecononfig 파일을 생성해야 합니다.

프라이빗 클러스터용 Astra Connector를 설치합니다

Astra Control Service는 Astra Connector를 사용하여 Astra Control Service와 프라이빗 클러스터 간의 통신을 지원한다. 관리할 프라이빗 클러스터에 Astra Connector를 설치해야 합니다.

Astra Connector는 다음과 같은 유형의 프라이빗 클러스터를 지원합니다.

- Amazon Elastic Kubernetes Service(EKS)
- Azure Kubernetes 서비스(AKS)
- Google Kubernetes Engine(GKE)
- AWS 기반 Red Hat OpenShift Service(ROSA)
- AWS PrivateLink와 함께 Rosa
- Red Hat OpenShift Container Platform 온-프레미스

Astra Connector를 설치합니다

이 작업에 대해

- 이러한 단계를 수행할 때 Astra Control Service로 관리할 프라이빗 클러스터에 대해 다음 명령을 실행합니다.
- 방호 호스트를 사용하는 경우 방호 호스트의 명령줄에서 이러한 명령을 실행하십시오.

시작하기 전에

- Astra Control Service를 사용하여 관리할 프라이빗 클러스터에 액세스해야 합니다.

- 클러스터에 Astra Connector 연산자를 설치하려면 Kubernetes 관리자 권한이 필요합니다.

단계

1. Astra Control Service로 관리할 프라이빗 클러스터에 Astra Connector 연산자를 설치합니다. 이 명령을 실행하면 네임스페이스가 생성됩니다 `astra-connector-operator` 이 만들어지고 구성이 네임스페이스에 적용됩니다.

```
kubectl apply -f https://github.com/NetApp/astra-connector-operator/releases/download/23.07.0-202310251519/astraconnector_operator.yaml
```

2. 작업자가 설치되어 있고 준비가 되었는지 확인합니다.

```
kubectl get all -n astra-connector-operator
```

3. Astra Control에서 API 토큰을 받습니다. 을 참조하십시오 "Astra 자동화 문서" 를 참조하십시오.
4. Astra-Connector 네임스페이스를 생성합니다.

```
kubectl create ns astra-connector
```

5. Astra Connector CR 파일을 생성하고 이름을 지정합니다 `astra-connector-cr.yaml`. 괄호 <> 의 값을 Astra Control 환경 및 클러스터 구성과 일치하도록 업데이트합니다.
   - * <ASTRA_CONTROL_SERVICE_URL> * : Astra Control Service의 웹 UI URL. 예를 들면 다음과 같습니다.

     ```
     https://astra.netapp.io
     ```

   - * <ASTRA_CONTROL_SERVICE_API_TOKEN> *: 이전 단계에서 얻은 Astra Control API 토큰.
   - * <PRIVATE_AKS_CLUSTER_NAME> *: (AKS 클러스터만 해당) - 전용 Azure Kubernetes Service 클러스터의 클러스터 이름입니다. 전용 AKS 클러스터를 추가하는 경우에만 주석을 해제하고 이 줄을 채웁니다.
   - * <ASTRA_CONTROL_ACCOUNT_ID> *: Astra Control 웹 UI에서 가져옵니다. 페이지 오른쪽 상단의 그림 아이콘을 선택하고 * API 액세스 * 를 선택합니다.

```yaml
apiVersion: netapp.astraconnector.com/v1
kind: AstraConnector
metadata:
  name: astra-connector
  namespace: astra-connector
spec:
  natssync-client:
    cloud-bridge-url: <ASTRA_CONTROL_SERVICE_URL>
  imageRegistry:
    name: theotw
    secret: ""
  astra:
    token: <ASTRA_CONTROL_SERVICE_API_TOKEN>
    #clusterName: <PRIVATE_AKS_CLUSTER_NAME>
    accountId: <ASTRA_CONTROL_ACCOUNT_ID>
    acceptEULA: yes
```

6. 를 채운 후 `astra-connector-cr.yaml` 올바른 값이 있는 파일에 CR을 적용합니다.

```
kubectl apply -f astra-connector-cr.yaml
```

7. Astra Connector가 완전히 구축되었는지 확인:

```
kubectl get all -n astra-connector
```

8. 클러스터가 Astra Control에 등록되었는지 확인:

```
kubectl get astraconnector -n astra-connector
```

다음과 유사한 출력이 표시됩니다.

```
NAME                REGISTERED   ASTRACONNECTORID
STATUS
astra-connector     true         be475ae5-1511-4eaa-9b9e-712f09b0d065
Registered with Astra
```

ASTRACONNECTORID를 기록해 둡니다. Astra Control에 클러스터를 추가할 때 필요합니다.

다음 단계

Astra Connector를 설치했으므로 이제 프라이빗 클러스터를 Astra Control Service에 추가할 준비가 되었습니다.

- "Astra Control Service에 프라이빗 공급자 관리 클러스터를 추가합니다": 다음 단계를 수행하여 프라이빗 IP 주소가 있고 클라우드 공급자가 관리하는 클러스터를 추가할 수 있습니다. 클라우드 공급자의 서비스 주 계정, 서비스 계정 또는 사용자 계정이 필요합니다.
- "Astra Control Service에 프라이빗 자체 관리 클러스터를 추가합니다": 다음 단계를 사용하여 개인 IP 주소가 있고 조직에서 관리하는 클러스터를 추가할 수 있습니다. 추가하려는 클러스터에 대해 kubecononfig 파일을 생성해야 합니다.

를 참조하십시오

- "클러스터를 추가합니다"

공급자 관리 클러스터를 추가합니다

공용 공급자 관리 클러스터를 **Astra Control Service**에 추가합니다

클라우드 환경을 설정한 후에는 Kubernetes 클러스터를 생성하고 Astra Control Service에 추가할 준비가 된 것입니다.

- Kubernetes 클러스터를 생성합니다
- 클러스터를 Astra Control Service에 추가합니다
- 기본 스토리지 클래스를 변경합니다

Kubernetes 클러스터를 생성합니다

클러스터가 아직 없는 경우 요건에 맞는 클러스터를 생성할 수 있습니다 "Amazon Elastic Kubernetes Service(EKS)에 대한 Astra Control 서비스 요구사항". 클러스터가 아직 없는 경우 요건에 맞는 클러스터를 생성할 수 있습니다 "Google Kubernetes Engine(GKE)에 대한 Astra Control Service 요구 사항". 클러스터가 아직 없는 경우 요건에 맞는 클러스터를 생성할 수 있습니다 "Azure NetApp Files를 사용하는 Azure Kubernetes Service(AKS)에 대한 Astra Control 서비스 요구사항" 또는 "Azure 관리 디스크를 사용하는 Azure Kubernetes Service(AKS)에 대한 Astra Control Service 요구 사항".

> Astra Control Service는 인증 및 ID 관리를 위해 Azure Active Directory(Azure AD)를 사용하는 AKS 클러스터를 지원합니다. 클러스터를 생성할 때 의 지침을 따릅니다 "공식 문서" Azure AD를 사용하도록 클러스터를 구성합니다. 클러스터가 AKS로 관리되는 Azure AD 통합에 대한 요구 사항을 충족하는지 확인해야 합니다.

클러스터를 **Astra Control Service**에 추가합니다

Astra Control Service에 로그인한 후 첫 번째 단계는 클러스터 관리를 시작하는 것입니다. Astra Control Service에 클러스터를 추가하려면 먼저 특정 작업을 수행하고 클러스터가 특정 요구 사항을 충족하는지 확인해야 합니다.

Azure Kubernetes Service 및 Google Kubernetes Engine 클러스터를 관리할 때 Astra Control Provisioner 설치 및 라이프사이클 관리에 대한 두 가지 옵션이 있습니다.

- Astra Control Service를 사용하여 Astra Control Provisioner의 라이프사이클을 자동으로 관리할 수 있습니다. 이렇게 하려면 Astra Trident가 설치되어 있지 않고 Astra Control Service를 사용하여 관리하려는 클러스터에 Astra Control Provisioner가 활성화되어 있지 않아야 합니다. 이 경우 클러스터 관리를 시작할 때 Astra Control Service가 Astra Control Provisioner를 자동으로 활성화하며 Astra Control Provisioner 업그레이드는 자동으로 처리됩니다.
- Astra Control Provisioner의 라이프사이클을 직접 관리할 수 있습니다. 이렇게 하려면 Astra Control Service로 클러스터를 관리하기 전에 클러스터에서 Astra Control Provisioner를 활성화합니다. 이 경우 Astra Control Service는 Astra Control Provisioner가 이미 활성화되었음을 감지하여 Astra Control Provisioner 업그레이드를 다시 설치하거나 관리하지 않습니다. 을 참조하십시오 "Astra Control Provisioner를 활성화합니다" 단계를 위해 Astra Control Provisioner를 활성화합니다.

Astra Control Service를 사용하여 Amazon Web Services 클러스터를 관리할 때 Astra Control Provisioner에서만 사용할 수 있는 스토리지 백엔드가 필요한 경우 Astra Control Service를 통해 클러스터를 관리하기 전에 클러스터에서 Astra Control Provisioner를 수동으로 활성화해야 합니다. 을 참조하십시오 "Astra Control Provisioner를 활성화합니다" Astra Control Provisioner를 활성화하는 단계에 대해 알아보십시오.

시작하기 전에

Amazon Web Services에서 직접 지원합니다

- 클러스터를 생성한 IAM 사용자의 자격 증명이 포함된 JSON 파일이 있어야 합니다. "IAM 사용자를 생성하는 방법을 알아봅니다".
- Amazon FSx for NetApp ONTAP에는 Astra Control Provisioner가 필요합니다. Amazon FSx for NetApp ONTAP를 EKS 클러스터의 스토리지 백엔드로 사용할 계획인 경우 에서 Astra Control Provisioner 정보를 참조하십시오 "EKS 클러스터 요구 사항".
- (선택 사항) 제공해야 하는 경우 `kubectl` 클러스터 생성자가 아닌 다른 IAM 사용자에 대한 클러스터에 대한 명령 액세스는 의 지침을 참조하십시오 "Amazon EKS에서 클러스터를 생성한 후 다른 IAM 사용자 및 역할에 대한 액세스를 제공하려면 어떻게 해야 합니까?".
- NetApp Cloud Volumes ONTAP를 스토리지 백엔드로 사용하려는 경우 Amazon Web Services와 연동되도록 Cloud Volumes ONTAP를 구성해야 합니다. Cloud Volumes ONTAP를 참조하십시오 "설치 설명서".

단계

1. (선택 사항) Amazon EKS 클러스터를 추가하거나 Astra Control Provisioner의 설치 및 업그레이드를 직접 관리하려는 경우 클러스터에서 Astra Control Provisioner를 활성화합니다. 을 참조하십시오 "Astra Control Provisioner를 활성화합니다" 참조하십시오.
2. 브라우저에서 Astra Control Service 웹 UI를 엽니다.
3. 대시보드에서 * Kubernetes 클러스터 관리 * 를 선택합니다.

   표시되는 메시지에 따라 클러스터를 추가합니다.

4. * 공급자 *: 클라우드 공급자를 선택한 다음 새 클라우드 인스턴스를 생성하는 데 필요한 자격 증명을 제공하거나 사용할 기존 클라우드 인스턴스를 선택하십시오.
5. * Amazon Web Services *: JSON 파일을 업로드하거나 클립보드에서 해당 JSON 파일의 콘텐츠를 붙여넣어 Amazon Web Services IAM 사용자 계정에 대한 세부 정보를 제공합니다.

   JSON 파일에는 클러스터를 생성한 IAM 사용자의 자격 증명이 포함되어야 합니다.

6. * 기타 *: 이 탭은 자체 관리형 클러스터에만 사용됩니다.

a. * 클라우드 인스턴스 이름 *: 이 클러스터를 추가할 때 생성될 새 클라우드 인스턴스의 이름을 입력합니다. 에 대해 자세히 알아보십시오 "클라우드 인스턴스".

b. 다음 * 을 선택합니다.

   Astra Control Service에는 선택할 수 있는 클러스터 목록이 표시됩니다.

c. * 클러스터 *: 목록에서 Astra Control Service에 추가할 클러스터를 선택합니다.

   > 클러스터 목록에서 선택할 때는 * 자격 조건 * 열에 주의해야 합니다. 클러스터가 "부적격" 또는 "부분적으로 적격"인 경우 상태 위로 마우스를 가져가면 클러스터에 문제가 있는지 확인할 수 있습니다. 예를 들어 클러스터에 작업자 노드가 없는 것을 식별할 수 있습니다.

d. 다음 * 을 선택합니다.

e. (선택 사항) * 스토리지 *: 선택적으로 이 클러스터에 Kubernetes 애플리케이션을 배포할 스토리지 클래스를 선택하여 기본적으로 사용하도록 합니다.

7. 클러스터에 대한 새 기본 스토리지 클래스를 선택하려면 * 새 기본 스토리지 클래스 할당 * 확인란을 설정합니다.
8. 목록에서 새 기본 스토리지 클래스를 선택합니다.

   > 각 클라우드 공급자의 스토리지 서비스에는 다음과 같은 가격, 성능 및 복원력 정보가 표시됩니다.
   >
   > - Amazon Elastic Block Store: 가격, 성능 또는 복원력 정보를 사용할 수 없습니다
   > - NetApp ONTAP용 Amazon FSx: 가격, 성능 또는 복원력 정보 없음
   > - NetApp Cloud Volumes ONTAP: 가격, 성능 또는 복원력 정보를 제공할 수 없습니다

   각 스토리지 클래스는 다음 서비스 중 하나를 활용할 수 있습니다.

   - "Amazon Elastic Block Store를 클릭합니다"
   - "NetApp ONTAP용 Amazon FSx"
   - "NetApp Cloud Volumes ONTAP를 참조하십시오"

     에 대해 자세히 알아보십시오 "Amazon Web Services 클러스터용 스토리지 클래스입니다". 에 대해 자세히 알아보십시오 "AKS 클러스터용 스토리지 클래스입니다". 에 대해 자세히 알아보십시오 "GKE 클러스터용 저장소 클래스".

     a. 다음 * 을 선택합니다.
     b. * 검토 및 승인 *: 구성 세부 정보를 검토합니다.
     c. 클러스터를 Astra Control Service에 추가하려면 * 추가 * 를 선택합니다.

결과

이 클라우드 공급자를 위해 추가한 첫 번째 클러스터인 경우 Astra Control Service는 해당 클러스터에서 실행되는 애플리케이션 백업을 위해 클라우드 공급자용 오브젝트 저장소를 생성합니다. (이 클라우드 공급자에 후속 클러스터를 추가할 경우 더 이상 오브젝트 저장소가 생성되지 않습니다.) 기본 스토리지 클래스를 지정한 경우 Astra Control Service는 사용자가 지정한 기본 스토리지 클래스를 설정합니다. Amazon Web Services 또는 Google Cloud Platform에서 관리되는 클러스터의 경우 Astra Control Service는 클러스터에 관리자 계정도 생성합니다. 이 작업은 몇 분 정도 걸릴 수 있습니다.

기본 스토리지 클래스를 변경합니다

클러스터의 기본 스토리지 클래스를 변경할 수 있습니다.

Astra Control을 사용하여 기본 스토리지 클래스를 변경합니다

Astra Control 내에서 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 클러스터에서 이전에 설치된 스토리지 백엔드 서비스를 사용하는 경우 이 방법을 사용하여 기본 스토리지 클래스를 변경하지 못할 수 있습니다(* 기본값으로 설정* 작업은 선택할 수 없음). 이 경우 를 사용할 수 있습니다 명령줄을 사용하여 기본 스토리지 클래스를 변경합니다.

단계

1. Astra Control Service UI에서 * Clusters * 를 선택합니다.
2. 클러스터 * 페이지에서 변경할 클러스터를 선택합니다.
3. Storage * 탭을 선택합니다.
4. 스토리지 클래스 * 범주를 선택합니다.
5. 기본값으로 설정할 스토리지 클래스에 대해 * Actions * 메뉴를 선택합니다.
6. Set as default * 를 선택합니다.

명령줄을 사용하여 기본 스토리지 클래스를 변경합니다

Kubernetes 명령을 사용하여 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 이 방법은 클러스터의 구성에 관계없이 작동합니다.

단계

1. Kubernetes 클러스터에 로그인합니다.
2. 클러스터의 스토리지 클래스를 나열합니다.

```
kubectl get storageclass
```

3. 기본 스토리지 클래스에서 기본 지정을 제거합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-
class":"false"}}}'
```

4. 다른 스토리지 클래스를 기본값으로 표시합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

5. 새 기본 스토리지 클래스를 확인합니다.

```
kubectl get storageclass
```